

POLITECHNIKA LUBELSKA
Wydział Elektrotechniki i Informatyki
Kierunek Inżynierskie zastosowania
informatyki w elektrotechnice



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

**Bazy danych SQL – projekt bazy
danych słowni**

Mikołaj Kończyński
Zbigniew Kobyliński
Jakub Kondraciuk

Lublin, 2025

Spis treści

1.	OPIS PROJEKTU	3
2.	PROJEKTOWANIE STRUKTURY BAZY DANYCH.....	4
3.	TWORZENIE BAZY DANYCH W SYSTEMIE MYSQL	14
4.	TWORZENIE SKRYPTU WPROWADZAJĄCEGO DANE	16
5.	TWORZENIE ZAPYTAŃ BAZODANOWYCH	17
6.	TWORZENIE WIDOKÓW	20
7.	TWORZENIE WYZWALACZY	25
8.	POBIERANIA INFORMACJI O OBIEKTACH BAZODANOWYCH	27
9.	EKSPORT BAZY DANYCH	29
10.	PODSUMOWANIE	30

1. Opis projektu

Projekt dotyczy systemu informatycznego do zarządzania prywatną siłownią. Siłownia oferuje różnorodne usługi, takie jak sprzedaż karnetów o różnych typach, treningi personalne prowadzone przez wykwalifikowanych pracowników (trenerów), a także dostęp do dodatkowych atrakcji, takich jak sauna czy sprzęt treningowy. System ma na celu wspomaganie zarządzania klientami, pracownikami, sprzętem, płatnościami oraz usługami dodatkowymi, co pozwala na sprawną obsługę i kontrolę działalności siłowni. W zakresie funkcjonalności system umożliwia zarządzanie pracownikami i ich stanowiskami, pozwalając na dodawanie, modyfikowanie i usuwanie danych personalnych wraz z przypisaniem do konkretnych ról, takich jak trener personalny, recepcjonista czy manager. System rejestruje również klientów, przechowując ich dane osobowe i kontaktowe oraz przypisując im zakupione karnety o różnych typach, takich jak miesięczny, kwartalny, półroczny czy roczny. Prowadzi ewidencję płatności związanych z karnetami oraz treningami personalnymi, umożliwiając kontrolę historii zakupów i rozliczeń. Dodatkowo, system pozwala na planowanie i rejestrowanie treningów personalnych, powiązanych z klientami i trenerami, zawierających szczegóły dotyczące daty, godziny, kwoty oraz statusu zakończenia treningu. Zarządzanie sprzętem siłowni obejmuje ewidencję dostępnych urządzeń, opisów, dat zakupu oraz odpowiedzialnych pracowników. System rejestruje również awarie sprzętu wraz z terminami zgłoszenia i naprawy, co umożliwia utrzymanie wysokiej jakości oferowanych usług. Kontrola dostępu do siłowni i sauny realizowana jest poprzez zapisywanie dat i godzin wejść klientów, co pozwala na monitorowanie frekwencji oraz popularności poszczególnych usług. System zapewnia także możliwość generowania raportów i analiz dotyczących aktywności klientów, efektywności pracy pracowników, historii płatności oraz stanu sprzętu. Całość rozwiązania jest skonstruowana tak, aby umożliwić elastyczne zarządzanie ofertą siłowni, zachowując szczegółowe informacje o klientach i pracownikach, co sprzyja sprawnej komunikacji i organizacji pracy. Dzięki rejestracji awarii i napraw sprzętu możliwe jest szybkie reagowanie na problemy techniczne, a rejestracja wejść wspiera analizę zachowań użytkowników oraz optymalizację dostępnych usług.

2. Projektowanie struktury bazy danych

1. Wyłonienie listy tabel.

- *Klienci*
- *Pracownicy*
- *Treningi personalne*
- *Karnety*
- *Płatności*
- *Wejścia siłownia*
- *Wejścia sauna*
- *Sprzęt*
- *Awarie sprzętu*
- *Stanowiska (encja)*
- *Typy karnetów (encja)*
- *Typy płatności (encja)*

2. Opis kolejnych tabel

Nazwa tabeli	Typ	Opis
<i>Klienci</i>	<i>tabela danych</i>	Przechowuje dane klientów korzystających z siłowni, takie jak imię, nazwisko, email, numer telefonu oraz data urodzenia.
<i>Pracownicy</i>	<i>tabela danych</i>	Zawiera informacje o pracownikach zatrudnionych w siłowni, w tym ich dane osobowe (imię, nazwisko, email, telefon, data urodzenia) oraz identyfikator stanowiska.
<i>Treningi Personalne</i>	<i>tabela danych</i>	Zawiera dane dotyczące indywidualnych treningów personalnych, takie jak daty rozpoczęcia i zakończenia, kwota, identyfikator klienta oraz prowadzącego pracownika.
<i>Karnety</i>	<i>tabela danych</i>	Przechowuje informacje o wykupionych karnetach przez klientów – identyfikator klienta, data zakupu, kwota oraz typ karnetu.
<i>Płatności</i>	<i>tabela danych</i>	Rejestruje informacje o płatnościach dokonanych przez klientów, w tym datę, identyfikator klienta oraz typ płatności.
<i>Wejścia siłownia</i>	<i>tabela danych</i>	Zawiera dane o wejściach klientów na siłownię – data wejścia oraz identyfikator klienta.
<i>Wejścia sauna</i>	<i>tabela danych</i>	Rejestruje wejścia klientów do sauny – data wejścia oraz identyfikator klienta.
<i>Sprzęt</i>	<i>tabela danych</i>	Przechowuje informacje o sprzęcie dostępnym na siłowni – nazwa, opis, data zakupu oraz identyfikator pracownika odpowiedzialnego za sprzęt.
<i>Awarie sprzętu</i>	<i>tabela danych</i>	Zawiera dane dotyczące awarii sprzętu – opis awarii, data wystąpienia i naprawy, identyfikator uszkodzonego sprzętu.
<i>Stanowiska</i>	tabela walidacyjna	Przechowuje listę możliwych stanowisk pracowników w siłowni – np. trener, recepcjonista, serwisant.
<i>Typy karnetów</i>	tabela walidacyjna	Zawiera predefiniowane typy karnetów dostępnych w ofercie siłowni, np. miesięczny, kwartalny, roczny.
<i>Typy płatności</i>	tabela walidacyjna	Przechowuje listę typów płatności, np. gotówka, karta, przelew bankowy.

3. Wyłonienie pól poszczególnych tabel

Reprezentacja tabeli Klienci

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idKlienta</i> (id)	całkowity (INT)	TAK	TAK	Wewnętrzny identyfikator klienta – klucz główny tabeli.
<i>imię</i>	znakowe (VARCHAR(50))	TAK	NIE	Imię klienta.
<i>nazwisko</i>	znakowe (VARCHAR(50))	TAK	NIE	Nazwisko klienta.
<i>email</i>	znakowe (VARCHAR(100))	TAK	TAK	Adres e-mail klienta.
<i>nrTelefonu</i>	znakowe (VARCHAR(15))	TAK	TAK	Numer telefonu klienta.
<i>dataUrodzenia</i>	data (DATE)	TAK	NIE	Data urodzenia klienta.

Reprezentacja tabeli Pracownicy

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idPracownika</i>	całkowity (INT)	TAK	TAK	Wewnętrzny identyfikator klienta – klucz główny tabeli.
<i>imię</i>	znakowe (VARCHAR(50))	TAK	NIE	Imię pracownika.
<i>nazwisko</i>	znakowe (VARCHAR(50))	TAK	NIE	Nazwisko pracownika.
<i>email</i>	znakowe (VARCHAR(100))	TAK	TAK	Adres e-mail pracownika.
<i>nrTelefonu</i>	znakowe (VARCHAR(15))	TAK	TAK	Numer telefonu pracownika.

<i>dataUrodzenia</i>	data (DATE)	TAK	NIE	Data urodzenia pracownika.
<i>idStanowiska</i>	całkowity (INT)	TAK	NIE	Odwołanie do stanowiska pracownika.

Reprezentacja tabeli TreningiPersonalne

Nazwa pola	Typ pola	Czy pole jest wymagane?	Czy wartość jest unikatowa?	Opis
<i>idTreningu</i>	całkowity (INT)	TAK	TAK	Wewnętrzny identyfikator klienta – klucz główny tabeli.
<i>dataRozpoczecia</i>	data i czas (DATETIME)	TAK	NIE	Data i czas rozpoczęcia treningu.
<i>dataZakonczenia</i>	znakowe (VARCHAR(50))	TAK	NIE	Data i czas zakończenia treningu.
<i>kwota</i>	liczbowy (DECIMAL(8,2))	TAK	NIE	Kwota zapłacona za trening.
<i>idKlienta</i>	całkowity (INT)	TAK	NIE	Identyfikator klienta biorącego udział w treningu.
<i>idPracownika</i>	całkowity (INT)	TAK	NIE	Identyfikator trenera prowadzącego trening.

Reprezentacja tabeli Karnety

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idKarnetu</i>	całkowity (INT)	TAK	TAK	Identyfikator karnetu.
<i>idKlienta</i>	całkowity (INT)	TAK	NIE	Identyfikator klienta posiadającego karnet.
<i>dataZakupu</i>	data (DATE)	TAK	NIE	Data zakupu karnetu.
<i>kwota</i>	liczbowy (DECIMAL(8,2))	TAK	NIE	Kwota zapłacona za karnet.
<i>idTypu</i>	całkowity (INT)	TAK	NIE	Identyfikator typu karnetu.

Reprezentacja tabeli Płatności

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idPlatnosci</i>	całkowity (INT)	TAK	TAK	Identyfikator płatności.
<i>idKlienta</i>	całkowity (INT)	TAK	NIE	Identyfikator klienta, który dokonał płatności.
<i>dataPlatnosci</i>	data i czas (DATETIME)	TAK	NIE	Data i czas dokonania płatności.
<i>idTypu</i>	całkowity (INT)	TAK	NIE	Identyfikator typu płatności.

Reprezentacja tabeli WejściaSiłownia

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idWejscia</i>	całkowity (INT)	TAK	TAK	Identyfikator płatności.
<i>dataWejscia</i>	data i czas (DATETIME)	TAK	NIE	Data i czas wejścia klienta na siłownię
Klient_idKlienta	całkowity (INT)	TAK	NIE	Identyfikator klienta.

Reprezentacja tabeli WejściaSauna

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idWejscia</i>	całkowity (INT)	TAK	TAK	Identyfikator wejścia do sauny.
<i>dataWejscia</i>	data i czas (DATETIME)	TAK	NIE	Data i czas wejścia do sauny.
Klient_idKlienta	całkowity (INT)	TAK	NIE	Identyfikator klienta.

Reprezentacja tabeli Sprzęt

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idSprzetu</i>	całkowity (INT)	TAK	TAK	Identyfikator sprzętu.
<i>nazwa</i>	znakowe (VARCHAR(50))	TAK	NIE	Nazwa urządzenia.
<i>opis</i>	znakowe (VARCHAR(50))	TAK	NIE	Krótki opis sprzętu.
<i>dataZakupu</i>	data (DATE)	TAK	NIE	Data zakupu sprzętu.

<i>idPracownika</i>	całkowity (INT)	TAK	NIE	Identyfikator pracownika odpowiedzialnego za sprzęt.
---------------------	-----------------	-----	-----	--

Reprezentacja tabeli AwarieSprzętu

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idAwarii</i>	całkowity (INT)	TAK	TAK	Identyfikator awarii.
<i>opis</i>	znakowe (VARCHAR(100))	TAK	NIE	Opis awarii sprzętu.
<i>dataAwarii</i>	data i czas (DATETIME)	TAK	NIE	Data wystąpienia awarii.
<i>dataNaprawy</i>	data i czas (DATETIME)	NIE	NIE	Data naprawy sprzętu (jeśli naprawiony)
<i>idSprzetu</i>	całkowity (INT)	TAK	NIE	Identyfikator uszkodzonego sprzętu.

Reprezentacja tabeli Stanowiska

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idTypu</i>	całkowity (INT)	TAK	TAK	Identyfikator stanowiska.
<i>nazwa</i>	znakowe (VARCHAR(45))	TAK	TAK	Nazwa stanowiska pracownika.

Reprezentacja tabeli typyKarnetów

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idTypu</i>	całkowity (INT)	TAK	TAK	Identyfikator typu karnetu.
<i>nazwa</i>	znakowe (VARCHAR(45))	TAK	TAK	Nazwa typu karnetu, np. miesięczny, roczny.

Reprezentacja tabeli typyPłatności

<i>Nazwa pola</i>	<i>Typ pola</i>	<i>Czy pole jest wymagane?</i>	<i>Czy wartość jest unikatowa?</i>	<i>Opis</i>
<i>idTypu</i>	całkowity (INT)	TAK	TAK	Identyfikator typu płatności.
<i>nazwa</i>	znakowe (VARCHAR(45))	TAK	TAK	Nazwa rodzaju płatności, np. gotówka, karta.

4. Definiowanie kluczy w tabelach

<i>Nazwa tabeli</i>	<i>Nazwa klucza głównego</i>
Klienci	IdKlienta
Pracownik	IdPracownika
TreningiPersonalne	idTreningu
Karnety	idKarnetu
Płatności	idPlatnosci
WejściaSilownia	idWejscia
WejściaSauna	idWejscia
Sprzęt	idSprzetu
AwarieSprzętu	idAwarii
Stanowiska	idTypu
typyKarnetow	idTypu
typyPlatnosci	idTypu

Klucze obce dla wszystkich tabel.

Klucze obce dla tabeli Pracownicy

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Stanowiska	idStanowiska

Klucze obce dla tabeli TreningiPersonalne

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Klienci	idKlienta
Pracownicy	idPracownika

Klucze obce dla tabeli Karnety

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Klienci	idKlienta
typyKarnetów	idTypu

Klucze obce dla tabeli Płatności

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Klienci	idKlienta
typyPłatności	idTypu

Klucze obce dla tabeli WejściaSiłownia

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Klienci	Klient_idKlienta

Klucze obce dla tabeli WejściaSauna

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Klienci	Klient_idKlienta

Klucze obce dla tabeli Sprzęt

<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Pracownicy	idPracownika

Klucze obce dla tabeli AwarieSprzętu

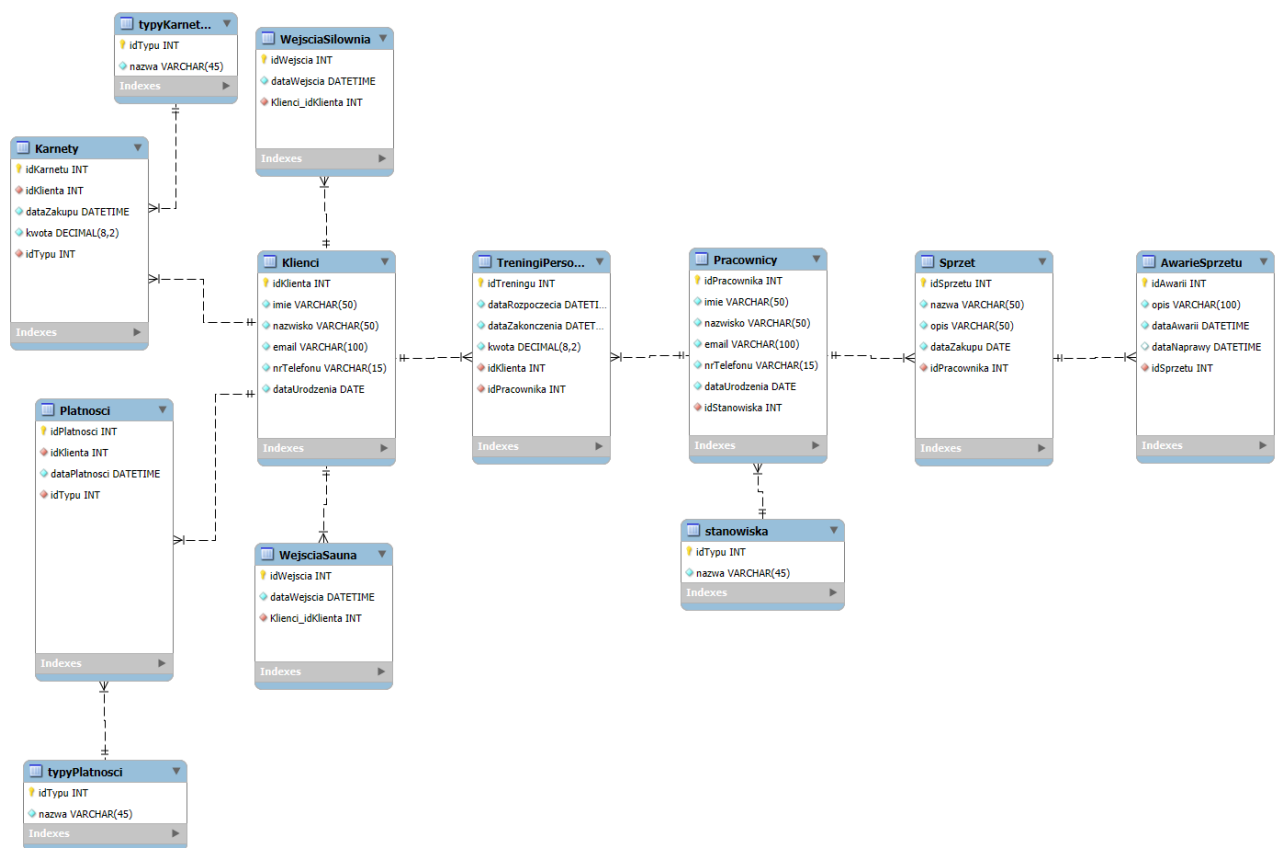
<i>Nazwa tabeli pierwotnej</i>	<i>Nazwa klucza obcego</i>
Sprzet	idSprzętu

5. Identyfikowanie relacji tabel

Tabela relacji pomiędzy tabelami

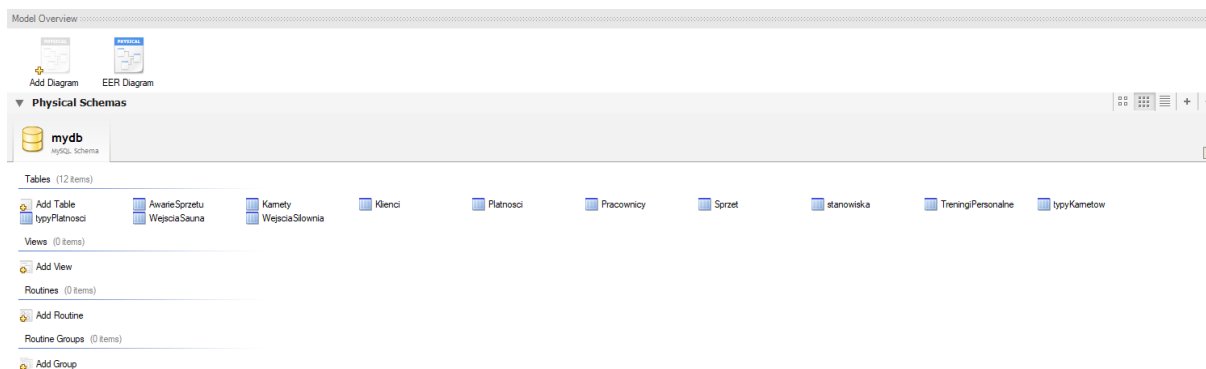
	Klienci	Pracownicy	TreningiPersonalne	Karnety	Płatności	WejsciaSiłownia	WejsciaSauna	Sprzet	AwarieSprzetu	Stanowiska	typyKarnetow	typyPłatnosci
Klienci			N:1	N:1	N:1	N:1	N:1					
Pracownicy			N:1					N:1		1:N		
TreningiPersonalne	1:N	1:N										
Karnety	1:N										N:1	
Płatności	1:N											N:1
WejsciaSiłownia	1:N											
WejsciaSauna	1:N											
Sprzet		1:N							N:1			
AwarieSprzetu								1:N				
Stanowiska		1:N										
typyKarnetow				1:N								
typyPłatnosci					1:N							

6. Zaprojektowanie struktury bazy danych

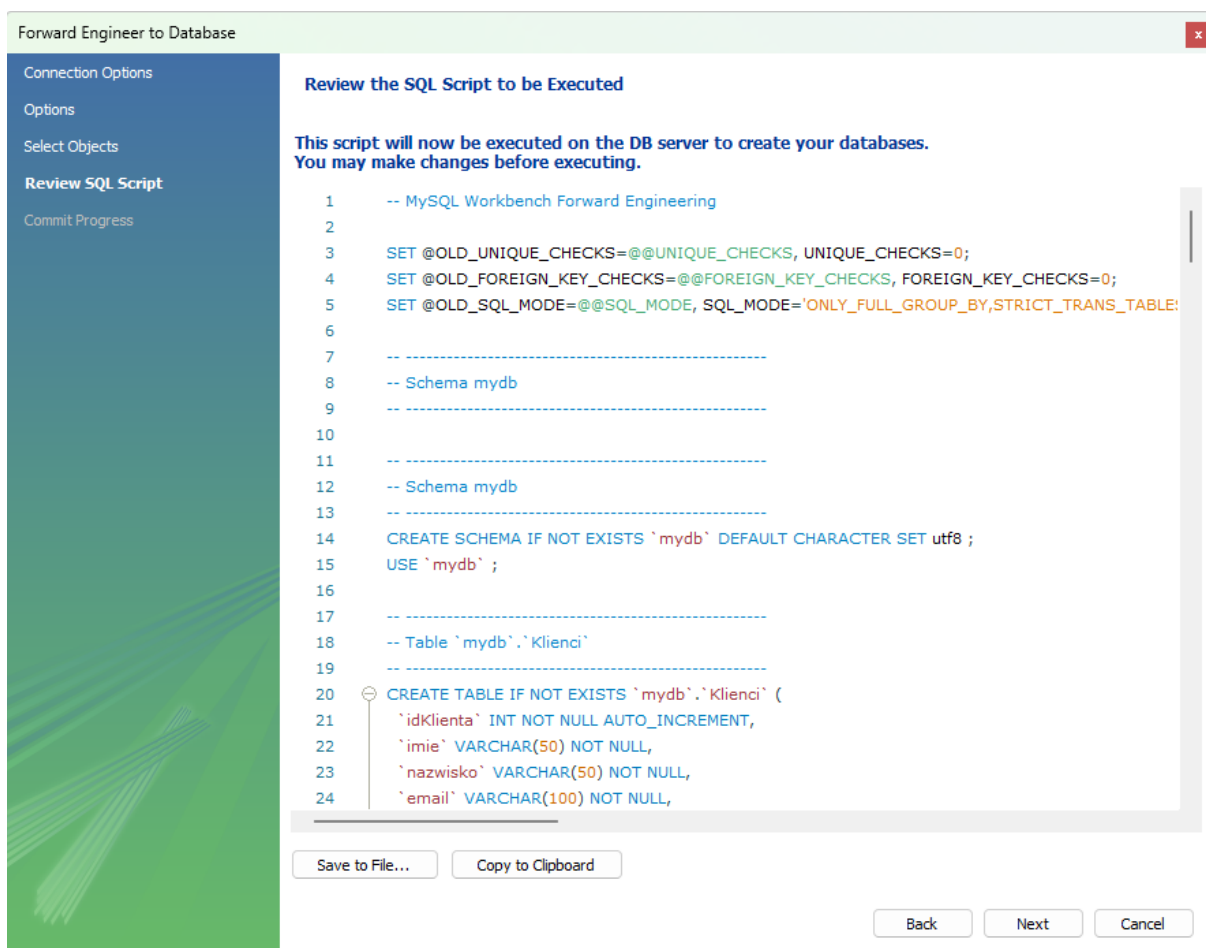


Rys.1 Diagram EER

3. Tworzenie bazy danych w systemie MySQL



Rys.2 Utworzona baza danych



Rys.3 Wygenerowany skrypt

db

MySQL Workbench Forward Engineering

```

1  -- MySQL Workbench Forward Engineering
2
3  • SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
4  • SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
5  • SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
6
7  -----
8  -- Schema silownia4
9  -----
10 • CREATE SCHEMA IF NOT EXISTS `silownia4` DEFAULT CHARACTER SET utf8 ;
11 • USE `silownia4` ;
12
13 -----
14 -- Table `Klienci`
15 -----
16 • CREATE TABLE IF NOT EXISTS `Klienci` (
17   `idKlienta` INT NOT NULL AUTO_INCREMENT,
18   `imie` VARCHAR(50) NOT NULL,
19   `nazwisko` VARCHAR(50) NOT NULL,
20   `email` VARCHAR(100) NOT NULL,
21   `nrTelefonu` VARCHAR(15) NOT NULL,
22   `dataUrodzenia` DATE NOT NULL,
23   PRIMARY KEY (`idKlienta`),
24   UNIQUE INDEX `idKlienta_UNIQUE` (`idKlienta`),
25   UNIQUE INDEX `email_UNIQUE` (`email`),
26   UNIQUE INDEX `nrTelefonu_UNIQUE` (`nrTelefonu`)

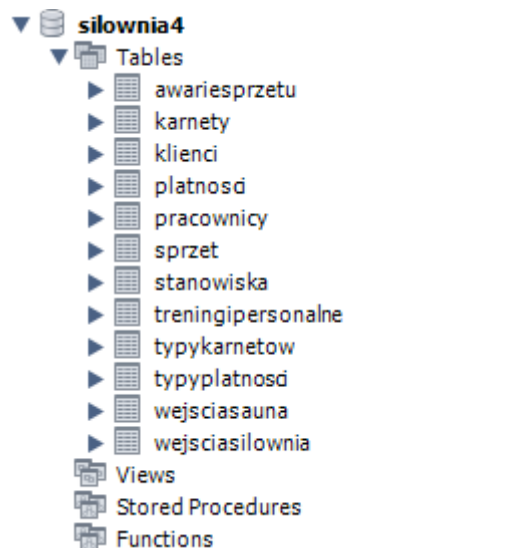
```

Output

Action Output

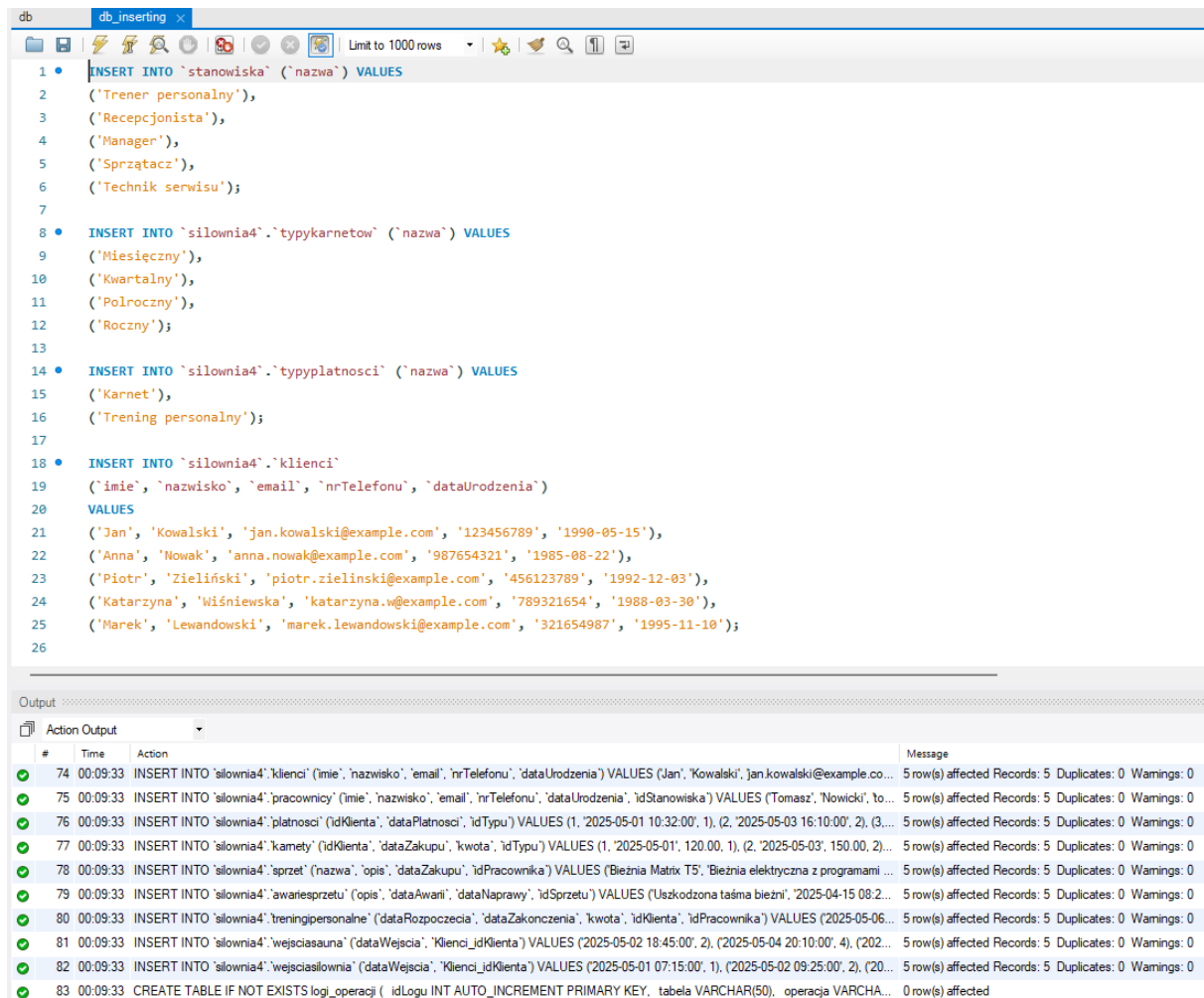
#	Time	Action	Message
61	00:06:52	CREATE TABLE IF NOT EXISTS `typyPlatnosci` (`idTypu` INT NOT NULL AUTO_INCREMENT, `nazwa` VARCHAR(45) NOT NULL, PRIM...	0 row(s) affected
62	00:06:52	CREATE TABLE IF NOT EXISTS `Platnosci` (`idPlatnosci` INT NOT NULL AUTO_INCREMENT, `idKlienta` INT NOT NULL, `dataPlatnosci` ...	0 row(s) affected
63	00:06:52	CREATE TABLE IF NOT EXISTS `Sprzet` (`idSprzetu` INT NOT NULL AUTO_INCREMENT, `nazwa` VARCHAR(50) NOT NULL, `opis` VAR...	0 row(s) affected
64	00:06:52	CREATE TABLE IF NOT EXISTS `AwarieSprzetu` (`idAwarii` INT NOT NULL AUTO_INCREMENT, `opis` VARCHAR(100) NOT NULL, `data...	0 row(s) affected
65	00:06:52	CREATE TABLE IF NOT EXISTS `WejsciaSilownia` (`idWejscia` INT NOT NULL AUTO_INCREMENT, `dataWejscia` DATETIME NOT NULL...	0 row(s) affected
66	00:06:52	CREATE TABLE IF NOT EXISTS `WejsciaSauna` (`idWejscia` INT NOT NULL AUTO_INCREMENT, `dataWejscia` DATETIME NOT NULL, ...	0 row(s) affected
67	00:06:52	CREATE TABLE IF NOT EXISTS `TreningiPersonalne` (`idTreningu` INT NOT NULL AUTO_INCREMENT, `dataRozpoczecia` DATETIME N...	0 row(s) affected
68	00:06:52	SET SQL_MODE=@OLD_SQL_MODE	0 row(s) affected
69	00:06:52	SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS	0 row(s) affected
70	00:06:52	SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS	0 row(s) affected

Rys.4 Wykonanie skryptu



Rys.5 Utworzone tabele

4. Tworzenie skryptu wprowadzającego dane



The screenshot displays a database management interface with a SQL script editor and an output window. The script consists of several INSERT statements for a database named 'silownia4'.

```
1 • INSERT INTO `stanowiska` (`nazwa`) VALUES
2   ('Trener personalny'),
3   ('Recepcjonista'),
4   ('Manager'),
5   ('Sprzątac'),
6   ('Technik serwisu');
7
8 • INSERT INTO `silownia4`.`typykartow` (`nazwa`) VALUES
9   ('Miesięczny'),
10  ('Kwartalny'),
11  ('Półroczny'),
12  ('Roczny');
13
14 • INSERT INTO `silownia4`.`typyplatnosci` (`nazwa`) VALUES
15   ('Karnet'),
16   ('Trening personalny');
17
18 • INSERT INTO `silownia4`.`klienci`
19   (`imie`, `nazwisko`, `email`, `nrTelefonu`, `dataUrodzenia`)
20   VALUES
21   ('Jan', 'Kowalski', 'jan.kowalski@example.com', '123456789', '1990-05-15'),
22   ('Anna', 'Nowak', 'anna.nowak@example.com', '987654321', '1985-08-22'),
23   ('Piotr', 'Zieliński', 'piotr.zielinski@example.com', '456123789', '1992-12-03'),
24   ('Katarzyna', 'Wiśniewska', 'katarzyna.w@example.com', '789321654', '1988-03-30'),
25   ('Marek', 'Lewandowski', 'marek.lewandowski@example.com', '321654987', '1995-11-10');
26
```

The output window shows the execution results of the script, including the number of rows affected, duplicates, and warnings for each statement.

#	Time	Action	Message
74	00:09:33	INSERT INTO `silownia4`.`klienci` (`imie`, `nazwisko`, `email`, `nrTelefonu`, `dataUrodzenia`) VALUES ('Jan', 'Kowalski', 'jan.kowalski@example.co...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
75	00:09:33	INSERT INTO `silownia4`.`pracownicy` (`imie`, `nazwisko`, `email`, `nrTelefonu`, `dataUrodzenia`, `idStanowiska`) VALUES ('Tomasz', 'Nowicki', 'to...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
76	00:09:33	INSERT INTO `silownia4`.`platnosci` (`idKlienta`, `dataPlatnosci`, `idTypu`) VALUES (1, '2025-05-01 10:32:00', 1), (2, '2025-05-03 16:10:00', 2), (3...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
77	00:09:33	INSERT INTO `silownia4`.`kamety` (`idKlienta`, `dataZakupu`, `kwota`, `idTypu`) VALUES (1, '2025-05-01', 120.00, 1), (2, '2025-05-03', 150.00, 2)...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
78	00:09:33	INSERT INTO `silownia4`.`sprzet` (`nazwa`, `opis`, `dataZakupu`, `idPracownika`) VALUES ('Bieżnia Matrix T5', 'Bieżnia elektryczna z programami ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
79	00:09:33	INSERT INTO `silownia4`.`awariesprzetu` (`opis`, `dataAwarii`, `dataNaprawy`, `idSprzetu`) VALUES ('Uszkodzona taśma bieżni', '2025-04-15 08:2...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
80	00:09:33	INSERT INTO `silownia4`.`treningipersonalne` (`dataRozpoczecia`, `dataZakonczenia`, `kwota`, `idKlienta`, `idPracownika`) VALUES ('2025-05-06...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
81	00:09:33	INSERT INTO `silownia4`.`wejsciasauna` (`dataWejscia`, `Klienci_idKlienta`) VALUES ('2025-05-02 18:45:00', 2), ('2025-05-04 20:10:00', 4), ('202...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
82	00:09:33	INSERT INTO `silownia4`.`wejsciasilownia` (`dataWejscia`, `Klienci_idKlienta`) VALUES ('2025-05-01 07:15:00', 1), ('2025-05-02 09:25:00', 2), ('20...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
83	00:09:33	CREATE TABLE IF NOT EXISTS logi_operacji (`idLogu` INT AUTO_INCREMENT PRIMARY KEY, `tabela` VARCHAR(50), `operacja` VARCHA...	0 row(s) affected

Rys.6 Skrypt wprowadzający dane

5. Tworzenie zapytań bazodanowych

Należy wykonać po 5 zapytań bazodanowych:

- proste;

1. Lista wszystkich klientów

```
SELECT * FROM klienci;
```

idKlienta	imie	nazwisko	email	nrTelefonu	dataUrodzenia
1	Jan	Kowalski	jan.kowalski@example.com	123456789	1990-05-15
2	Anna	Nowak	anna.nowak@example.com	987654321	1985-08-22
3	Piotr	Zieliński	piotr.zielinski@example.com	456123789	1992-12-03
4	Katarzyna	Wiśniewska	katarzyna.w@example.com	789321654	1988-03-30
5	Marek	Lewandowski	marek.lewandowski@example.com	321654987	1995-11-10

2. Imiona i nazwiska pracowników

```
SELECT imie, nazwisko FROM pracownicy;
```

imie	nazwisko
Tomasz	Nowicki
Ewa	Kubiak
Adam	Maj
Karolina	Zając
Michał	Lis

3. Nazwy typów kartetów

```
SELECT nazwa FROM typykartetow;
```

nazwa
Kwartalny
Miesięczny
Polroczny
Roczny

4. Dane sprzętu zakupionego po 2023-01-01

```
SELECT nazwa, dataZakupu FROM sprzet WHERE dataZakupu > '2023-01-01';
```

nazwa	dataZakupu
Bieżnia Matrix T5	2024-01-10
Atlas Kettler	2023-11-22
Ławka do wyciskania	2023-08-05
Wiosłomistrz X200	2024-02-01

5. Lista klientów urodzonych przed 1990 rokiem

```
SELECT imie, nazwisko, dataUrodzenia FROM klienci WHERE dataUrodzenia < '1990-01-01';
```

imie	nazwisko	dataUrodzenia
Anna	Nowak	1985-08-22
Katarzyna	Wiśniewska	1988-03-30

- złożone;

1. Lista treningów personalnych z nazwami klientów i trenerów

```
SELECT t.dataRozpoczenia, t.dataZakonczenia, t.kwota, concat(k.imie, " ", k.nazwisko) klient, concat(p.imie, " ", p.nazwisko) trener
FROM treningipersonalne t
JOIN klienci k ON t.idKlienta = k.idKlienta
JOIN pracownicy p ON t.idPracownika = p.idPracownika;
```

dataRozpoczenia	dataZakonczenia	kwota	klient	trener
2025-05-06 08:00:00	2025-05-06 09:00:00	100.00	Jan Kowalski	Tomasz Nowicki
2025-05-08 17:00:00	2025-05-08 18:00:00	100.00	Piotr Zieliński	Tomasz Nowicki
2025-05-10 10:30:00	2025-05-10 11:30:00	90.00	Marek Lewandowski	Tomasz Nowicki
2025-05-12 14:00:00	2025-05-12 15:00:00	110.00	Anna Nowak	Tomasz Nowicki
2025-05-14 09:00:00	2025-05-14 10:00:00	95.00	Katarzyna Wiśniewska	Tomasz Nowicki

2. Suma wydatków na treningi personalne każdego klienta

```
SELECT k.imie, k.nazwisko, SUM(t.kwota) AS suma
FROM klienci k
JOIN treningipersonalne t ON k.idKlienta = t.idKlienta
GROUP BY k.idKlienta;
```

imie	nazwisko	suma
Jan	Kowalski	100.00
Anna	Nowak	110.00
Piotr	Zieliński	100.00
Katarzyna	Wiśniewska	95.00
Marek	Lewandowski	90.00

3. Sprzęt i nazwa pracownika odpowiedzialnego za jego opiekę

```
SELECT s.nazwa sprzet, concat(p.imie, " ", p.nazwisko) pracownik
FROM sprzet s
JOIN pracownicy p ON s.idPracownika = p.idPracownika;
```

sprzet	pracownik
Bieżnia Matrix T5	Michał Lis
Atlas Kettler	Michał Lis
Rower stacjonarny BH	Michał Lis
Ławka do wyciskania	Michał Lis
Wiosłomistrz X200	Michał Lis

4. Liczba sprzętów przypisanego do każdego pracownika

```
SELECT p.imie, p.nazwisko, COUNT(s.idSprzetu) liczbaSprzetow
FROM pracownicy p
LEFT JOIN sprzet s ON p.idPracownika = s.idPracownika
GROUP BY p.idPracownika;
```

imie	nazwisko	liczbaSprzetow
Tomasz	Nowicki	0
Ewa	Kubiak	0
Adam	Maj	0
Karolina	Zajac	0
Michał	Lis	5

5. Czas działania sprzętu do pierwszej awarii

```
SELECT s.nazwa, DATEDIFF(a.dataAwarii, s.dataZakupu) dniDoPierwszejAwarii
FROM sprzet s
JOIN awariesprzetu a ON s.idSprzetu = a.idSprzetu
WHERE a.dataAwarii = (
    SELECT MIN(dataAwarii) FROM awariesprzetu WHERE idSprzetu = s.idSprzetu
);
```

nazwa	dniDoPierwszejAwarii
Bieżnia Matrix T5	461
Atlas Kettler	474
Rower stacjonarny BH	813
Ławka do wyciskania	563
Wiosłomistrz X200	455

- z podzapytaniem.

1. Klienci, którzy zapłacili za trening personalny

```
SELECT imie, nazwisko FROM klienci
WHERE idKlienta IN (
    SELECT idKlienta FROM platnosci WHERE idTypu = 2
);
```

imie	nazwisko
Anna	Nowak
Marek	Lewandowski

2. Klienci, którzy korzystali z sauny po 2025-05-05

```
SELECT * FROM klienci
WHERE idKlienta IN (
    SELECT Klienci_idKlienta FROM wejsciasauna WHERE dataWejscia > '2025-05-05'
);
```

idKlienta	imie	nazwisko	email	nrTelefonu	dataUrodzenia
2	Anna	Nowak	anna.nowak@example.com	987654321	1985-08-22
5	Marek	Lewandowski	marek.lewandowski@example.com	321654987	1995-11-10

3. Trenerzy, którzy prowadzili treningi personalne

```
SELECT * FROM pracownicy
WHERE idPracownika IN (
    SELECT DISTINCT idPracownika FROM treningipersonalne
);
```

idPracownika	imie	nazwisko	email	nrTelefonu	dataUrodzenia	idStanowiska
1	Tomasz	Nowicki	tomasz.nowicki@silownia.pl	500600700	1980-07-20	1

4. Najdroższy trening personalny i klient, który go odbył

```
SELECT t.idTreningu, k.imie, k.nazwisko, t.kwota
FROM treningipersonalne t
JOIN klienci k ON t.idKlienta = k.idKlienta
WHERE t.kwota = (
    SELECT MAX(kwota) FROM treningipersonalne
);
```

idTreningu	imie	nazwisko	kwota
4	Anna	Nowak	110.00

5. Klienci, którzy mają karnety typu kwartalnego

```
SELECT * FROM klienci
WHERE idKlienta IN (
    SELECT idKlienta FROM karnety WHERE idTypu = 2
);
```

idKlienta	imie	nazwisko	email	nrTelefonu	dataUrodzenia
2	Anna	Nowak	anna.nowak@example.com	987654321	1985-08-22
5	Marek	Lewandowski	marek.lewandowski@example.com	321654987	1995-11-10

6. Tworzenie widoków

Należy utworzyć po 5 widoków:

- prostych;

1. Informacje o sprzęcie

Name:

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_sprzet_info` AS
6     SELECT
7         `silownia5`.`sprzet`.`idSprzetu` AS `idSprzetu`,
8         `silownia5`.`sprzet`.`nazwa` AS `nazwa`,
9         `silownia5`.`sprzet`.`dataZakupu` AS `dataZakupu`
10    FROM
11        `silownia5`.`sprzet`
```

2. Klienci, którzy mają kartę

Name: The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_karnety_klienci` AS
6     SELECT
7         `k`.`idKlienta` AS `idKlienta`,
8         `c`.`imie` AS `imie`,
9         `c`.`nazwisko` AS `nazwisko`,
10        `k`.`dataZakupu` AS `dataZakupu`,
11        `k`.`kwota` AS `kwota`
12    FROM
13        (`silownia5`.`karnety` `k`
14     JOIN `silownia5`.`klienci` `c` ON (`k`.`idKlienta` = `c`.`idKlienta`))
```

3. Kontakt do klientów

Name: The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_klienci_kontakt` AS
6     SELECT
7         `silownia5`.`klienci`.`idKlienta` AS `idKlienta`,
8         `silownia5`.`klienci`.`imie` AS `imie`,
9         `silownia5`.`klienci`.`nazwisko` AS `nazwisko`,
10        `silownia5`.`klienci`.`email` AS `email`,
11        `silownia5`.`klienci`.`nrTelefonu` AS `nrTelefonu`
12    FROM
13        `silownia5`.`klienci`
```

4. Wejścia na siłownię

Name: The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_wejscia_silownia` AS
6     SELECT
7         `w`.`Klienci_idKlienta` AS `Klienci_idKlienta`,
8         `k`.`imie` AS `imie`,
9         `k`.`nazwisko` AS `nazwisko`,
10        `w`.`dataWejscia` AS `dataWejscia`
11    FROM
12        (`silownia5`.`wejsciasilownia` `w`
13     JOIN `silownia5`.`klienci` `k` ON (`w`.`Klienci_idKlienta` = `k`.`idKlienta`))
```

5. Stanowisko każdego klienta

Name:

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_pracownicy_stanowiska` AS
6     SELECT
7         `p`.`idPracownika` AS `idPracownika`,
8         `p`.`imie` AS `imie`,
9         `p`.`nazwisko` AS `nazwisko`,
10        `s`.`nazwa` AS `stanowisko`
11    FROM
12        (`silownia5`.`pracownicy` `p`
13     JOIN `silownia5`.`stanowiska` `s` ON (`p`.`idStanowiska` = `s`.`idTypu`))
```

- złożonych.

1. Klienci, którzy mają karnet lub kupili trening personalny

Name:

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_aktywni_klienci` AS
6     SELECT DISTINCT
7         `k`.`idKlienta` AS `idKlienta`,
8         `k`.`imie` AS `imie`,
9         `k`.`nazwisko` AS `nazwisko`
10    FROM
11        `silownia5`.`klienci` `k`
12    WHERE
13        `k`.`idKlienta` IN (SELECT
14                            `silownia5`.`karnety`.`idKlienta`
15                        FROM
16                            `silownia5`.`karnety`)
17        AND `k`.`idKlienta` IN (SELECT
18                                `silownia5`.`treningipersonalne`.`idKlienta`
19                            FROM
20                                `silownia5`.`treningipersonalne`)
```

2. Czas naprawy sprzętu

Name:

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_sredni_czas_naprawy` AS
6     SELECT
7         `silownia5`.`awariesprzetu`.`idSprzetu` AS `idSprzetu`,
8         ROUND(AVG(TIMESTAMPDIFF(HOUR,
9             `silownia5`.`awariesprzetu`.`dataAwarii`,
10            `silownia5`.`awariesprzetu`.`dataNaprawy`)),
11            2) AS `sredni_czas_h`
12     FROM
13         `silownia5`.`awariesprzetu`
14     GROUP BY `silownia5`.`awariesprzetu`.`idSprzetu`
```

3. Statystyka wejść na siłownię i na saunę

Name:

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_statystyka_wejsc` AS
6     SELECT
7         `k`.`idKlienta` AS `idKlienta`,
8         `k`.`imie` AS `imie`,
9         `k`.`nazwisko` AS `nazwisko`,
10        (SELECT
11            COUNT(0)
12        FROM
13            `silownia5`.`wejsciasilownia` `ws`
14        WHERE
15            `ws`.`Klienci_idKlienta` = `k`.`idKlienta`) AS `wejsc_silownia`,
16        (SELECT
17            COUNT(0)
18        FROM
19            `silownia5`.`wejsciasauna` `wsau`
20        WHERE
21            `wsau`.`Klienci_idKlienta` = `k`.`idKlienta`) AS `wejsc_sauna`
22     FROM
23         `silownia5`.`klienci` `k`
```

4. Suma treningów każdego klienta

Name: `widok_suma_treningow_klienta`

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_suma_treningow_klienta` AS
6     SELECT
7         `k`.`idKlienta` AS `idKlienta`,
8         `k`.`imie` AS `imie`,
9         `k`.`nazwisko` AS `nazwisko`,
10        SUM(`t`.`kwota`) AS `suma_treningow`
11    FROM
12        (`silownia5`.`treningipersonalne` `t`
13     JOIN `silownia5`.`klienci` `k` ON (`t`.`idKlienta` = `k`.`idKlienta`))
14    GROUP BY `k`.`idKlienta`
```

5. Ostatnie wejścia każdego klienta

Name: `widok_ostatnie_wejscie_klientow`

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `silownia5`.`widok_ostatnie_wejscie_klientow` AS
6     SELECT
7         `k`.`idKlienta` AS `idKlienta`,
8         `k`.`imie` AS `imie`,
9         `k`.`nazwisko` AS `nazwisko`,
10        MAX(`w`.`dataWejscia`) AS `ostatnie_wejscie`
11    FROM
12        (`silownia5`.`klienci` `k`
13     JOIN `silownia5`.`wejsciasilownia` `w` ON (`k`.`idKlienta` = `w`.`Klienci_idKlienta`))
14    GROUP BY `k`.`idKlienta`
```


7. Tworzenie wyzwalaczy

Tabela do logów operacji (jeśli nie istnieje)

```
CREATE TABLE IF NOT EXISTS logi_operacji (  
    idLogu INT AUTO_INCREMENT PRIMARY KEY,  
    tabela VARCHAR(50),  
    operacja VARCHAR(20),  
    czas DATETIME,  
    szczegoly TEXT  
);
```

Należy utworzyć po 2 wyzwalacze (przed i po) wydarzeniu:

- **wprowadzenia danych;**

1. AFTER INSERT ON klienci - logowanie dodania klienta

```
CREATE TRIGGER after_insert_klienci  
AFTER INSERT ON klienci  
FOR EACH ROW  
BEGIN  
    INSERT INTO logi_operacji (tabela, operacja, czas, szczegoly)  
    VALUES ('klienci', 'INSERT', NOW(), CONCAT('Dodano klienta: ', NEW.imie, ' ', NEW.nazwisko));  
END$$
```

2. BEFORE INSERT ON klienci - walidacja numeru telefonu

```
CREATE TRIGGER before_insert_klienci  
BEFORE INSERT ON klienci  
FOR EACH ROW  
BEGIN  
    IF NEW.nrTelefonu NOT REGEXP '^[0-9]{9}$' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Numer telefonu musi mieć dokładnie 9 cyfr.';  
    END IF;  
END$$
```

- **usuwania danych;**

3. BEFORE DELETE ON pracownicy - blokowanie usuwania Managerów

```
CREATE TRIGGER before_delete_pracownicy  
BEFORE DELETE ON pracownicy  
FOR EACH ROW  
BEGIN  
    IF OLD.idStanowiska = (SELECT idStanowiska FROM stanowiska WHERE nazwa = 'Manager') THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Nie można usunąć pracownika o stanowisku Manager.';  
    END IF;  
END$$
```

4. AFTER DELETE ON pracownicy - logowanie usunięcia pracownika

```
CREATE TRIGGER after_delete_pracownicy
AFTER DELETE ON pracownicy
FOR EACH ROW
BEGIN
    INSERT INTO logi_operacji (tabela, operacja, czas, szczegoly)
    VALUES ('pracownicy', 'DELETE', NOW(), CONCAT('Usunięto: ', OLD.imie, ' ', OLD.nazwisko));
END$$
```

- modyfikowania danych.

5. BEFORE UPDATE ON treningipersonalne - walidacja kwoty treningu

```
CREATE TRIGGER before_update_treningi
BEFORE UPDATE ON treningipersonalne
FOR EACH ROW
BEGIN
    IF NEW.kwota < 50 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Kwota nie może być mniejsza niż 50 zł.';
    END IF;
END$$
```

6. AFTER UPDATE ON treningipersonalne - logowanie zmiany kwoty

```
CREATE TRIGGER after_update_treningi
AFTER UPDATE ON treningipersonalne
FOR EACH ROW
BEGIN
    INSERT INTO logi_operacji (tabela, operacja, czas, szczegoly)
    VALUES ('treningipersonalne', 'UPDATE', NOW(),
        CONCAT('Zmieniono kwotę z ', OLD.kwota, ' na ', NEW.kwota, ' dla klienta ID ', NEW.idKlienta));
END$$
```

Testowanie

```
INSERT INTO klienci (imie, nazwisko, email, nrTelefonu, dataUrodzenia)
VALUES ('Testowy', 'Klient', 'testowy1123@example.com', '123456712', '1990-01-01');
```

2	klienci	INSERT	2025-06-04 00:53:44	Dodano klienta: Testowy Klient
---	---------	--------	---------------------	--------------------------------

```
INSERT INTO klienci (imie, nazwisko, email, nrTelefonu, dataUrodzenia)
VALUES ('Zly', 'Numer', 'zly@example.com', '123', '1990-01-01');
```

✖ 121 00:56:42 INSERT INTO klienci (imie, nazwisko, email, nrTelefonu, dataUrodzenia) VALUES ('Zly', 'Numer', 'zly@example.com', '123', '1990-01-01') Error Code: 1644. Numer telefonu musi mieć dokładnie 9 cyfr.

```
UPDATE treningipersonalne SET kwota = 40 WHERE idKlienta = 1 LIMIT 1;
```

✖ 129 00:57:43 UPDATE treningipersonalne SET kwota = 40 WHERE idKlienta = 1 LIMIT 1 Error Code: 1644. Kwota nie może być mniejsza niż 50 zł.

```
UPDATE treningipersonalne SET kwota = 120 WHERE idKlienta = 1 LIMIT 1;
```

```
3      treningipersonalne  UPDATE    2025-06-04 00:58:09  Zmieniono kwotę z 100.00 na 120.00 dla klienta...
```

8. Pobierania informacji o obiektach bazodanowych

Należy napisać zapytania wyświetlające informacje o:

- utworzonych tabelach (polecenie describe lub zapytanie z information_schema);

```
SELECT TABLE_NAME, TABLE_TYPE
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'silownia4';
```

TABLE_NAME	TABLE_TYPE
awariesprzetu	BASE TABLE
karnety	BASE TABLE
klienci	BASE TABLE
logi_operacji	BASE TABLE
platnosci	BASE TABLE
pracownicy	BASE TABLE
sprzet	BASE TABLE
stanowiska	BASE TABLE
treningipersonalne	BASE TABLE
typykarnetow	BASE TABLE
typyplatnosci	BASE TABLE
wejsciasauna	BASE TABLE
wejsciasilownia	BASE TABLE

- utworzonych indeksach (polecenie show lub zapytanie z information_schema) ;

```
SELECT TABLE_NAME, INDEX_NAME, COLUMN_NAME, NON_UNIQUE
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'silownia4'
ORDER BY TABLE_NAME, INDEX_NAME;
```

TABLE_NAME	INDEX_NAME	COLUMN_NAME	NON_UNIQUE
awariesprzetu	email_UNIQUE	opis	0
awariesprzetu	fk_AwarieSprzetu_Sprzet1_idx	idSprzetu	1
awariesprzetu	idKlienta_UNIQUE	idAwarii	0
awariesprzetu	PRIMARY	idAwarii	0
karnety	fk_Karnety_Klienci_idx	idKlienta	1
karnety	fk_Karnety_typyKarnetow1_idx	idTypu	1
karnety	idKlienta_UNIQUE	idKarnetu	0
karnety	PRIMARY	idKarnetu	0
klienci	email_UNIQUE	email	0
klienci	idKlienta_UNIQUE	idKlienta	0
klienci	nrTelefonu_UNIQUE	nrTelefonu	0
klienci	PRIMARY	idKlienta	0
logi_operacji	PRIMARY	idLogu	0
platnosci	fk_Platnosci_Klienci1_idx	idKlienta	1
platnosci	fk_Platnosci_typyPlatnosci1_idx	idTypu	1
platnosci	idKlienta_UNIQUE	idPlatnosci	0
platnosci	PRIMARY	idPlatnosci	0
pracownicy	email_UNIQUE	email	0
pracownicy	fk_Pracownicy_stanowiska1_idx	idStanowiska	1
pracownicy	idKlienta_UNIQUE	idPracownika	0
pracownicy	nrTelefonu_UNIQUE	nrTelefonu	0
pracownicy	PRIMARY	idPracownika	0
sprzet	fk_Sprzet_Pracownicy1_idx	idPracownika	1
sprzet	idKlienta_UNIQUE	idSprzetu	0
sprzet	PRIMARY	idSprzetu	0
stanowiska	idTypu_UNIQUE	idTypu	0
stanowiska	nazwa_UNIQUE	nazwa	0
stanowiska	PRIMARY	idTypu	0
treningiperso...	fk_TreningiPersonalne_Klienci1...	idKlienta	1
treningiperso...	fk_TreningiPersonalne_Pracow...	idPracownika	1
treningiperso...	idKlienta_UNIQUE	idTreningu	0
treningiperso...	PRIMARY	idTreningu	0
typykarnetow	idTypu_UNIQUE	idTypu	0

- utworzonych widoków (polecenie show lub zapytanie z information_schema);

```
SELECT TABLE_NAME, VIEW_DEFINITION
```

```
FROM information_schema.VIEWS
```

```
WHERE TABLE_SCHEMA = 'silownia4';
```

TABLE_NAME	VIEW_DEFINITION
widok_aktywni_klienci	select distinct `k`.`idKlienta` AS `idKlienta`,`k...
widok_karnety_klienci	select `k`.`idKlienta` AS `idKlienta`,`c`.`imie`...
widok_klienci_kontakt	select `silownia5`.`klienci`.`idKlienta` AS `idKli...
widok_ostatnie_wejscie_klientow	select `k`.`idKlienta` AS `idKlienta`,`k`.`imie`...
widok_pracownicy_stanowiska	select `p`.`idPracownika` AS `idPracownika`,`...
widok_sprzet_info	select `silownia5`.`sprzet`.`idSprzetu` AS `id...
widok_sredni_czas_naprawy	select `silownia5`.`awariesprzetu`.`idSprzetu...
widok_statystyka_wejsc	select `k`.`idKlienta` AS `idKlienta`,`k`.`imie`...
widok_suma_treningow_klienta	select `k`.`idKlienta` AS `idKlienta`,`k`.`imie`...
widok_wejscia_silownia	select `w`.`Klienci_idKlienta` AS `Klienci_idKlie...

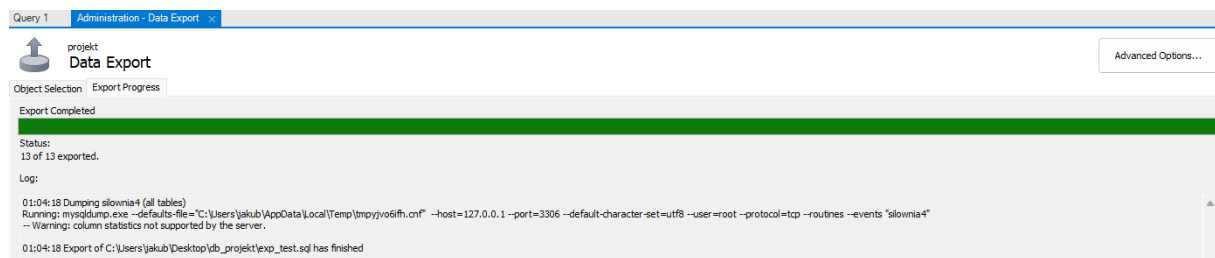
- utworzonych wyzwalaczy (polecenie show lub zapytanie z information_schema).

```
SELECT TRIGGER_NAME, EVENT_MANIPULATION, EVENT_OBJECT_TABLE, ACTION_TIMING
FROM information_schema.TRIGGERS
WHERE TRIGGER_SCHEMA = 'sloownia4';
```

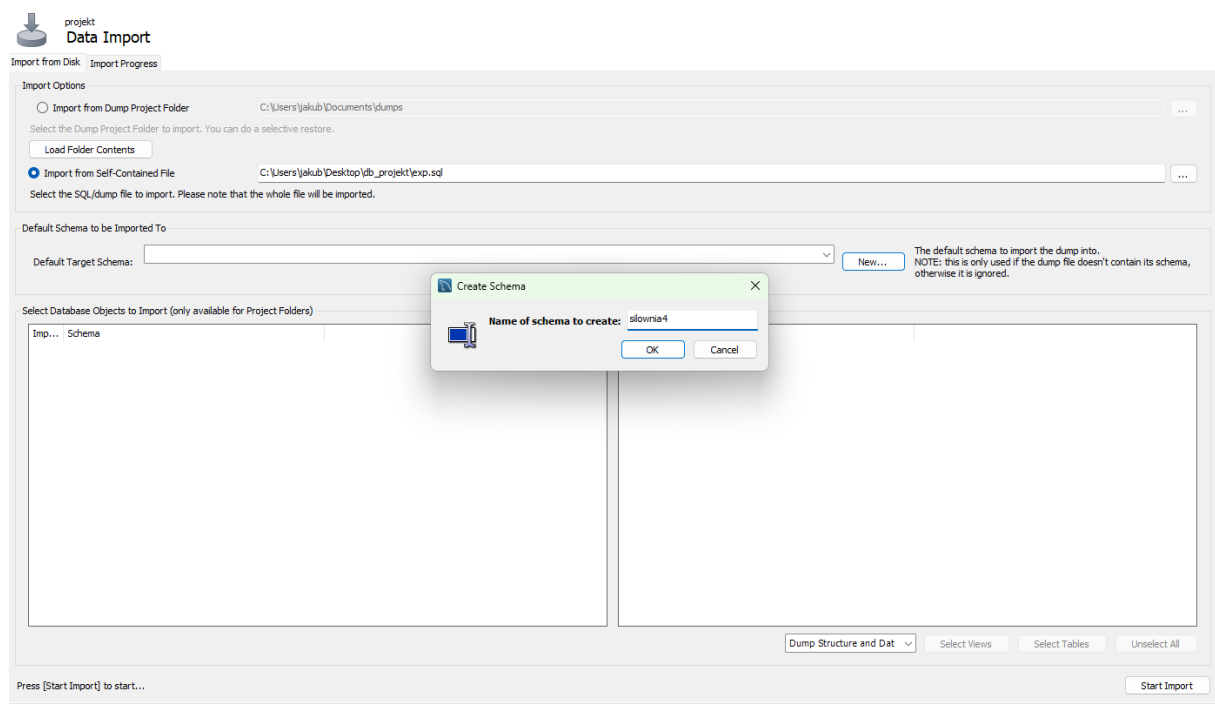
TRIGGER_NAME	EVENT_MANIPULATION	EVENT_OBJECT_TABLE	ACTION_TIMING
before_insert_klienci	INSERT	klienci	BEFORE
after_insert_klienci	INSERT	klienci	AFTER
before_delete_pracownicy	DELETE	pracownicy	BEFORE
after_delete_pracownicy	DELETE	pracownicy	AFTER
before_update_treningi	UPDATE	treningipersonalne	BEFORE
after_update_treningi	UPDATE	treningipersonalne	AFTER

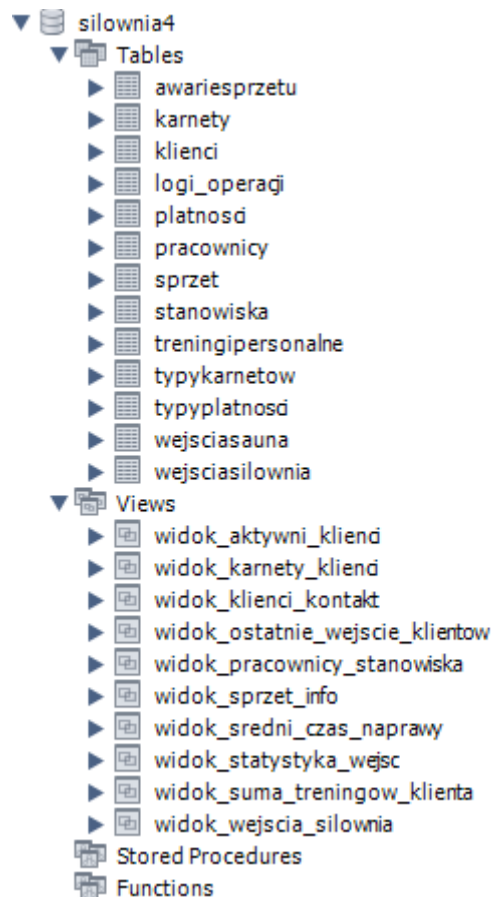
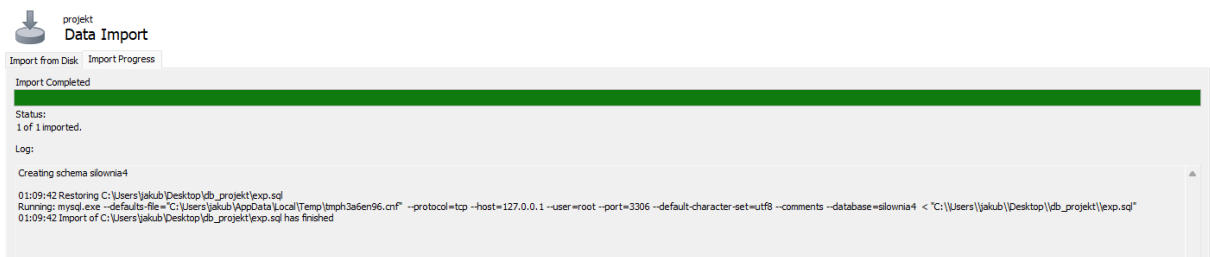
9. Eksport bazy danych

Wykonaliśmy eksport bazy danych jako jeden plik.



Wykonaliśmy import bazy danych





10. Podsumowanie

Projekt bazy danych dla siłowni obejmował kompleksowe rozwiązanie do zarządzania klientami, pracownikami, treningami personalnymi, karnetami, płatnościami, sprzętem oraz awariami. Został zrealizowany w systemie MySQL, a jego struktura została zaprojektowana w oparciu o diagram EER, który uwzględniał tabele, relacje oraz klucze główne i obce. W ramach projektu wykonano również skrypty wprowadzające dane, zapytania SQL (proste, złożone i z podzapytaniem), widoki, wyzwalacze oraz eksport i import bazy danych.